SV Randomization & Functional Coverage Assignment

Q1.

- Default branch is excluded from coverage. Repeat iteration increased for the functional coverage to hit 100%

```
import testing_pkg::*;

module tb();
  Transaction tr = new;
  byte operand1, operand2, out, expected_out;
  opcode_e opcode;
  bit clk, rst;
  int error_count;
  int correct_count;

  initial begin
    clk = 0;
    forever begin
      #1 clk = ~clk;
      tr.clk = clk;
    end
  end

  //<DUT Instantiation>
  alu_seq DUT(operand1, operand2, clk, rst, opcode, out);
```

```
initial begin
  assert_reset();
  repeat (1000) begin
    assert(tr.randomize);
    operand1 = tr.operand1;
    operand2 = tr.operand2;
    opcode = tr.opcode;
    golden_model(tr);
    @(negedge clk);
    check_result(expected_out);
  end
  assert_reset();

  $display("%0t: At end of test
  $stop;
end // initial begin
```

Karem Yasen

```
  task golden_model(input Transaction tr);
    expected_out = (tr.opcode == ADD)? tr.operand1 + tr.operand2 :
                   (tr.opcode == SUB)? tr.operand1 - tr.operand2 :
                   (tr.opcode == MULT)? tr.operand1 * tr.operand2 :
                   tr.operand1 / tr.operand2;
  endtask

  task check_result (input byte expected_result);
    @(negedge clk);
    if (expected_result != out) begin
      error_count = error_count + 1;
      $display("%t: Error: For opcode = %0d, operand1= 0d%0d, and op
    end
    else
      correct_count = correct_count + 1;
  endtask

  task assert_reset;
    //assert reset
    rst = 1;
    @(negedge clk);
    if (out !== 0) begin
      error_count = error_count + 1;
      $display("%t: Error: Reset value is asserted and the output is
    end
    else
      correct_count = correct_count + 1;

    //desert reset
    rst = 0;
  endtask
endmodule
```

Kareem Waseem

Q2.

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
COUNTER_1	When the reset is asserted, the output counter value should be low	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.	-	A checker in the testbench to make sure the output is correct
COUNTER_2	When the load is asserted, the output count_out should take the value of the load_data input	Randomization under constraints on the load signal to be off 70% of the time	Cover all values of load data	A checker in the testbench to make sure the output is correct
COUNTER_3	When the load/reset is deasserted, the output count_out increment as long as the enable is active and up_down is high	Randomization for up_down, and randomization for enable to be 70% of the time	Cover all values of count_out, and transition bin from max to zero	A checker in the testbench to make sure the output is correct
COUNTER_4	When the load/reset is deasserted, the output count_out decrement as long as the enable is active and up_down is low	Randomization for up_down, and randomization for enable to be 70% of the time	Cover all values of count_out, and transition bin from zero to max	A checker in the testbench to make sure the output is correct

```
package counter_pkg;
parameter W = 4;

class counter_transaction;
rand bit rst_n, load_n, up_down, ce;
rand bit [W-1:0] data_load;
bit [W-1:0] count_out;

constraint rst_con {
 rst_n dist {0:= 2, 1:= 98};
}

constraint load_con {
 load_n dist {0:= 70, 1:= 30};
}

constraint ce_con {
 ce dist {1:= 70, 0:= 30};
}

covergroup cg();
   data_load_cp: coverpoint data_load iff (rst_n && !load_n);
   count_bins_up_cp: coverpoint count_out iff (rst_n && ce && up_down);
   count_bins_up_trans_cp: coverpoint count_out iff (rst_n && ce && up_down) {
      bins max_zero_trans = ({W{1'b1}} => 0);
   }
   count_bins_dn_cp: coverpoint count_out iff (rst_n && ce && !up_down);
   count_bins_dn_trans_cp: coverpoint count_out iff (rst_n && ce && !up_down) {
      bins zero_max_trans = (0 => {W{1'b1}});
   }
endgroup : cg

function new();
   cg = new();
endfunction

endclass

endpackage
```

Karim Waseem

Q3.

Label	Description	Stimulus Generation	Functional Coverage	Functionality Check
ALSU_1	When the reset is asserted, the outputs should be low	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.	-	A checker in the testbench to make sure the output is correct
ALSU_2	Incase of invalid cases do not occur, when opcode is add, then out should perform the addition on ports A and B taking cin if parameter FULL_ADDER is high	Randomization under constraints on the A and B to have the maximum, minimum and zero values most of the time	Cover the maximum, minimum, and zero as well as default for other values of ports A and B	Output Checked against golden model
ALSU_3	Incase of invalid cases do not occur, when opcode is mult, then out should perform the multiplication on ports A and B	Randomization under constraints on the A and B to have the maximum, minimum and zero values most of the time	Cover the maximum, minimum, and zero as well as default for other values of ports A and B	Output Checked against golden model
ALSU_4	Incase of invalid cases do not occur, when opcode is OR, then out should perform the OR operation on ports A and B if reduction_A and reduction_B inputs are low	Randomization without constraints on A or B. reduction ports are disabled most of the time	-	Output Checked against golden model
ALSU_5	Incase of invalid cases do not occur, when opcode is OR & any of the inputs reduction_A or reduction_B is high, then out should perform the reduction OR operation on ports A and B based on the priority	Randomization under constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be low if reduction_A is high.	Cover the walking ones values for ports A and B	Output Checked against golden model
ALSU_6	Incase of invalid cases do not occur, when opcode is XOR, then out should perform the XOR operation on ports A and B if reduction_A and reduction_B inputs are low	Randomization without constraints on A or B. reduction ports are disabled most of the time	-	Output Checked against golden model
ALSU_7	Incase of invalid cases do not occur, when opcode is XOR & any of the inputs reduction_A or reduction_B is high, then out should perform the reduction XOR operation on ports A and B based on the priority	Randomization under constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be low if reduction_A is high	Cover the walking ones values for ports A and B	Output Checked against golden model
ALSU_8	Incase of invalid cases do not occur, when opcode is SHIFT then the output out will shift right or left based on the direction input	Randomization without constraints	Cover the SHIFT	Output Checked against golden model
ALSU_9	Incase of invalid cases do not occur, when opcode is ROTATE then the output out will rotate right or left based on the direction input	Randomization without constraints	Cover the ROTATE	Output Checked against golden model
ALSU_10	When invalid cases exist without bypass, out output should be low and leds should blink	Randomization under constraints where invalid cases do not occur as frequent as valid cases	Cover the illegal opcode values	Output Checked against golden model
ALSU_11	If invalid cases do not occur and the bypass inputs are high, then the output out should by bypass port A or B based on the prioirty if the both bypass ports are high	Randomization under constraint on bypass ports to be disabled most of the time	-	Output Checked against golden model
ALSU_12	In case of valid opcode values, the output should be evaluated based on the operation	Randomization under valid input stimulus while looping on different opcode every clock cycle	Cover the opcode values using legal, bins as well as transition to make sure output is evaluated at each opcode change with each transition	Output Checked against golden model

Kareem Waseem

```
package ALSU_pkg;
typedef enum bit [2:0] {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7} opcode_e;
typedef enum bit [2:0] {MAXPOS=3'b011, ZERO = 3'b000, MAXNEG=3'b100} reg_e;
parameter VALID_OP = 6;

class ALSU_inputs;
rand logic clk, rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
rand logic [2:0] A, B;
rand opcode_e opcode;
rand opcode_e opcode_valid [VALID_OP];
rand reg_e enum_ext_A, enum_ext_B;
rand logic  [3:0] A_rem_values, B_rem_values;
bit [2:0] walking_ones[] = '{3'b001, 3'b010, 3'b100};
rand bit [2:0] walking_ones_t, walking_ones_f;

// ALSU_1
constraint rst_con {
 rst dist {1:= 2, 0:= 98};
}

constraint A_B_con {
   A_rem_values != MAXPOS || 0 || MAXNEG;
   B_rem_values != MAXPOS || 0 || MAXNEG;
   walking_ones_t inside {walking_ones};
   !(walking_ones_f inside {walking_ones});

   if (opcode == OR || opcode == XOR) {
      // ALSU_5, ALSU_7
      if (red_op_A == 1) {
         B == 0;
         A dist {walking_ones_t:= 80, walking_ones_f:= 20};
      }  else if (red_op_B == 1) {
         A == 0;
         B dist {walking_ones_t:= 80, walking_ones_f:= 20};
      }
   } else {
      // ALSU_4, ALSU_6
      red_op_A dist {1:= 20, 0:= 80};
      red_op_B dist {1:= 20, 0:= 80};
      // ALSU_2, ALSU_3
      if (opcode == ADD || opcode == MULT) {
         A dist {enum_ext_A:= 80, A_rem_values:= 20};
         B dist {enum_ext_B:= 80, B_rem_values:= 20};
      }
   }
}

// ALSU_10
constraint opcode_con {
   opcode dist {[OR:ROTATE]:= 80, [INVALID_6:INVALID_7]:= 20};
}

// ALSU_11
constraint bypass_con {
   bypass_A dist {1:= 2, 0:= 98};
   bypass_B dist {1:= 2, 0:= 98};
}
```

Kareem Waseem

```
// ALSU_12
constraint opcode_seq_con {
   foreach (opcode_valid[i])
      foreach (opcode_valid[j]) {
         if (i != j) {
            opcode_valid[i] != opcode_valid[j];
            opcode_valid[i] inside {[OR:ROTATE]};
         }
      }
}

covergroup cvr_grp;
   //ALSU_2 ALSU_3
   A_cvp_values_ADD_MULT : coverpoint A {
      bins A_data_0 = {0};
      bins A_data_max = {MAXPOS};
      bins A_data_min = {MAXNEG};
      bins A_data_default = default;
   }
   //ALSU_5 ALSU_7
   A_cvp_values_RED : coverpoint A iff (red_op_A) {
      bins A_walkingones[] = {3'b001, 3'b010, 3'b100};
   }
   //ALSU_2 ALSU_3
   B_cvp_values_ADD_MULT : coverpoint B {
      bins B_data_0 = {0};
      bins B_data_max = {MAXPOS};
      bins B_data_min = {MAXNEG};
      bins B_data_default = default;
   }
   //ALSU_5 ALSU_7
   B_cvp_values_RED : coverpoint B iff (red_op_B && !red_op_A) {
      bins B_walkingones[] = {3'b001, 3'b010, 3'b100};
   }
   //ALSU_8 ALSU_9 ALSU_10 ALSU_12
   opcode_cvp_values : coverpoint opcode {
      bins bins_shift[] = {SHIFT, ROTATE};
      bins bins_arith[] = {ADD, MULT};
      bins bins_bitwise[] = {OR, XOR};
      illegal_bins opcode_invalid = {INVALID_6, INVALID_7};
      bins opcode_valid_trans = (OR => XOR => ADD => MULT => SHIFT => ROTATE);
   }
endgroup

function new();
   cvr_grp = new();
endfunction

endclass
endpackage
```

Kareem Waseem

```
import ALSU_pkg::*;

module ALSU_tb;
  parameter INPUT_PRIORITY = "A" ;
  parameter FULL_ADDER = "ON";
  bit clk ,rst ,cin ,red_op_A ,red_op_B ,bypass_A ,bypass_B ,direction ,serial_in;
  bit signed [2:0] A , B;
  opcode_e opcode;
  bit [15:0] leds_exp;
  logic [15:0] leds;
  logic signed [5:0] out;
  bit signed [5:0] out_exp;

  //Internals to register the stimulus for golden model
  bit red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
  bit signed [1:0] cin_reg;
  bit [2:0] opcode_reg;
  bit signed [2:0] A_reg, B_reg;

  int error_count = 0;
  int correct_count = 0;

  ALSU_inputs ALSU_obj = new();

  ALSU a1 (A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst,
```

```
// clock Generation
initial begin
  clk = 0 ;
  forever
  #1 clk = ~clk ;
end

initial begin
  assert_reset();
  ALSU_obj.opcode_seq_con.constraint_mode(0);
  repeat (10000)begin
    assert(ALSU_obj.randomize());
    cin = ALSU_obj.cin;
    red_op_A = ALSU_obj.red_op_A;
    red_op_B = ALSU_obj.red_op_B ;
    bypass_A = ALSU_obj.bypass_A;
    bypass_B = ALSU_obj.bypass_B;
    direction = ALSU_obj.direction;
    serial_in = ALSU_obj.serial_in;
    opcode = ALSU_obj.opcode;
    A = ALSU_obj.A;
    B = ALSU_obj.B;
    rst = ALSU_obj.rst;
    if (ALSU_obj.rst) begin
      check_rst();
      reset_internals();
    end
    else begin
```

```
      golden_model();
      check_results();
      if (!ALSU_obj.bypass_B || !ALSU_obj.bypass_A
        ALSU_obj.cvr_grp.sample();
    end
  end

ALSU_obj.constraint_mode(0);
ALSU_obj.opcode_seq_con.constraint_mode(1);
red_op_A = 0;
red_op_B = 0;
bypass_A = 0;
bypass_B = 0;
rst = 0;
repeat (10000) begin
   assert(ALSU_obj.randomize());
   cin = ALSU_obj.cin;
   direction = ALSU_obj.direction;
   serial_in = ALSU_obj.serial_in;
   A =  ALSU_obj.A;
   B = ALSU_obj.B;
   for (int i = 0; i < opcode.num()-2; i++) begin
      opcode = ALSU_obj.opcode_valid[i];
      ALSU_obj.opcode = ALSU_obj.opcode_valid[i];
      golden_model();
      check_results();
      ALSU_obj.cvr_grp.sample();
```

```
    end
  end

  $display("correct_count = %0d and error_count =%0d",correct_count , error_count);
  $stop;
end

task assert_reset();
  // reset asserted
  rst = 1;
  @(negedge clk);
  check_rst();
  rst = 0;
endtask

task check_rst();
  @(negedge clk);
  if (out != 0 || leds != 0) begin
    $display("Error in reset");
    error_count++;
  end
  else
    correct_count++;
  reset_internals();
endtask
```

Kareem Waseem

```
task reset_internals();
  {red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg} = 6'b00_0000;
  cin_reg = 2'b00;
  opcode_reg = 3'b00;
  {A_reg, B_reg} = 2'b00;
endtask

// check results task
task check_results();
  @(negedge clk);
    if ((out == out_exp) && (leds == leds_exp)) begin
      correct_count++;
    end
    else begin
      error_count++;
      $display("%0t : error when A =0d'%0d , B =0d'%0d , cin =0d'%0d, serial_in =0d'%0d, red_op_A =0d'%0d
      $display("leds =16'h%0h, leds_exp = 6'h%0h and  out =0d'%0d , out_exp = 0d'%0d ",leds , leds_exp ,
      $stop;
    end
 // end
endtask
```

```
// check for invalid cases function
function bit is_invalid();
  //invalid case 1
  if (opcode_reg == INVALID_6 || opcode_reg == INVALID_7)
    return 1;
  //invalid case 2
  else if ((opcode_reg > 3'b001) && (red_op_A_reg || red_op_B_reg))
    return 1;
  else
    return 0;
endfunction

task golden_model();
  //Handle leds_exp
  if(is_invalid())
    leds_exp = ~leds_exp;
  else
    leds_exp = 0;

  //Handle bypass, invalid and valid opcodes for out_exp

  // Check Bypass
  if (bypass_A_reg)
    out_exp = A_reg;
  else if (bypass_B_reg)
    out_exp = B_reg;

  //Check invalid
  else if (is_invalid())
    out_exp = 0;

  //Valid opcodes
  else begin
    // OR
    if (opcode_reg == OR) begin
      if (red_op_A_reg)
          out_exp = |A_reg;
        else if (red_op_B_reg)
          out_exp = |B_reg;
        else
          out_exp = A_reg | B_reg;
    end

    // XOR
    else if (opcode_reg == XOR) begin
      if (red_op_A_reg)
          out_exp = ^A_reg;
        else if (red_op_B_reg)
          out_exp = ^B_reg;
        else
```

```
            else
                out_exp = A_reg ^ B_reg;
        end

        // ADD
        else if (opcode_reg == ADD)
          out_exp = A_reg + B_reg + cin_reg;

        // MULT
        else if (opcode_reg == MULT)
          out_exp = A_reg * B_reg;

        // SHIFT
        else if (opcode_reg == SHIFT) begin
          if (direction_reg)
                out_exp = {out_exp[4:0], serial_in_reg};
            else
                out_exp = {serial_in_reg , out_exp[5:1]};
        end

        // ROTATE
        else if (opcode_reg == ROTATE) begin
          if (direction_reg)
                out_exp = {out_exp[4:0], out_exp[5]};
            else
                out_exp = {out_exp[0], out_exp[5:1]};
        end
      end
      update_internals();
    endtask

    task update_internals();
      cin_reg = cin;
      red_op_B_reg = red_op_B;
      red_op_A_reg = red_op_A;
      bypass_B_reg = bypass_B;
      bypass_A_reg = bypass_A;
      direction_reg = direction;
      serial_in_reg = serial_in;
      opcode_reg = opcode;
      A_reg = A;
      B_reg = B;
    endtask

  endmodule
```

Kareem Waseem

Q4. Bugs:

1- data_out output width should be 9 instead of 8.
2- Even parity is calculated instead of odd parity, the memory write statement should be mem_array[address] = {^data_in, data_in};

Label	Description	Stimulus Generation	Functional Coverage	Functionality Check
RAM_1	In case of the write port is high, data will be stored in the RAM in the address indicated by the address port. Also, even parity is calculated as the most sign bit on the RAM	write port is directed to be high. Randomized ports data and address without constraints	-	-
RAM_2	In case of the write port is low and read port is high, data_out will output the word stored in the RAM in the address indicated by the address port.	write port is directed to be low and read port is high. Randomized ports data and address without constraints	-	Output Checked against golden model

```
module RAM_tb;
// Dynamic array of addresses
bit [15:0] address_array [];
// Dynamic array of data to write
bit [7:0]  data_to_write_array[];
// Associative array of data expected to be read indexed by address
bit [8:0] data_read_expect_assoc[int];
// Queue of data read
logic [8:0] data_read_queue[$];

// Declare all inputs as bit
bit clk;
bit write;
bit read;
bit [7:0] data_in;
bit [15:0] address;

// Declare the output as 4-value logic
logic [8:0] data_out;

// # of memory locations to test
localparam  TESTS = 100;

// error_count and correct_count counters
int error_count = 0, correct_count = 0;

// Create a clock
initial begin
   forever #1 clk=~clk;
end

// Instantiate my_mem
my_mem my_mem(.*);

initial begin
write = 0;
read = 0;

// Allocate TESTS addresses for the address and data
address_array = new[TESTS];
data_to_write_array = new[TESTS];

stimulus_gen();
golden_model();
```

```
// ------ Do the test -------
// Do the writes
// RAM_1
write = 1;
for (int i=0;i<TESTS;i++) begin
   address = address_array[i];
   data_in = data_to_write_array[i];
   @(negedge clk);
end

// Do the reads in reverse order and check the results
// RAM_2
write = 0;
read = 1;
for (int i=0;i<TESTS;i++) begin
   address = address_array[i];
   Check9Bits(address);
   data_read_queue.push_back(data_out);
end

// Print out the read queue
read = 0;
$display("Elements in data_read_queue are:");
while (data_read_queue.size != 0)
  $displayh(data_read_queue.pop_front);

$display("%0t: At end of test error_count = 0d%0d and correct_count = 0d%0d", $time, error_count, corr
$stop;
end // initial begin


task Check9Bits(input bit [15:0] address);
   @(negedge clk);
   if (data_read_expect_assoc[address] !== data_out) begin
      $display("%t: Read from address 0x%0h expected 0x%0h but got 0x%0h", $time, address, data_read_e
      error_count = error_count + 1;
   end
   else
      correct_count = correct_count + 1;
endtask // Check9Bits

task stimulus_gen();
   // Fill the address & write dynamic arrays with random data addresses to write and then read
   for (int i=0;i<TESTS;i++) begin
      address_array[i] = $random;
      data_to_write_array[i] = $random;
   end
endtask

task golden_model();
   // Traverse the data to write dynamic array and create data_read_expect_assoc
   // Remember that the MSB is the even parity bit
   for (int i=0;i<TESTS;i++) begin
      data_read_expect_assoc[address_array[i]] = ({^data_to_write_array[i], data_to_write_array[i]});
   end
endtask
endmodule // test
```

Kareem Waseem